```
    distance_travelled = distance;
    callback();
  });

  this.Given(/^I am a (.*) Frequent Flyer member$/,
             function(status, callback) {
    traveller_status = status;
    callback();
  });

  this.When(/^I fly from (.*) to (.*)$/,
            function(start, end, callback) {
    callback();
  });

  this.Then(/^I should earn (\d+) travel points$/,
            function(expected_travel_points, callback) {
    var calculatedTravelPoints = calculate_points(traveller_status,
                                                  distance_travelled);
    calculatedTravelPoints.should.equal(parseInt(expected_travel_points))
    callback();
  });
};

module.exports = stepDefinitionWrapper;
```

Call callback() to tell Cucumber-JS that it can proceed to the next step.

Use the Chai assertion library to express your expectations.

This is a simple example to give you a feel for what BDD in JavaScript looks like. We'll look at some more advanced features of Cucumber-JS in the next chapter.

6.8.4 Running the scenarios

When you run Cucumber-JS feature files that have step implementations, Cucumber-JS will produce colored output on the console (see figure 6.13). You can also produce output in JSON format that can be used to import the results into other reporting tools such as Thucydides for more comprehensive reporting, or display them on a continuous integration server.

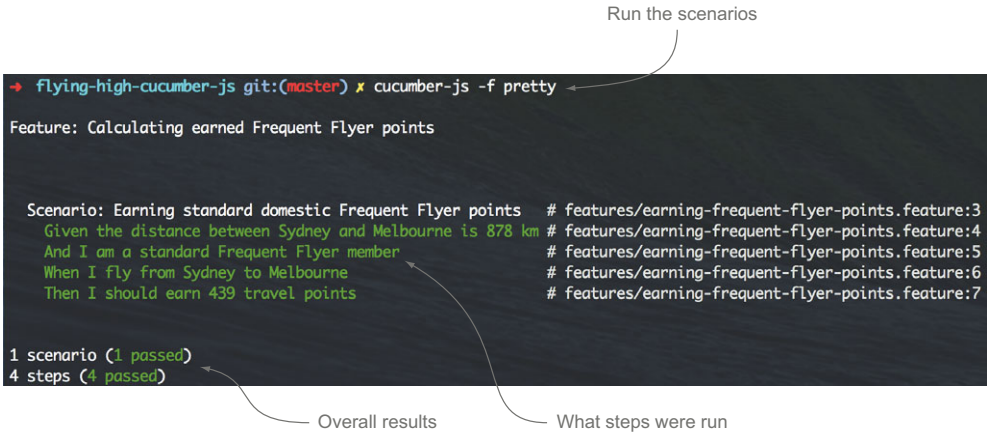


Figure 6.13 Running Cucumber-JS on the command line